

Credit Risk Assessment Using Ensemble Learning

Project Overview

This notebook builds a **Credit Risk Assessment System** that predicts the likelihood of loan default using a suite of ensemble machine learning models. Beyond raw classification, the system produces an interpretable **Risk Score (0–100)** and risk tier for each borrower — providing actionable intelligence suitable for financial institutions.

Scope of Analysis

- Exploratory Data Analysis & Feature Engineering
- Ensemble Models: Decision Tree, Bagging, AdaBoost, Random Forest, Gradient Boosting, XGBoost
- Unified Model Comparison (Accuracy, Precision, Recall, F1, ROC-AUC)
- ROC Curve Comparison
- Feature Importance & SHAP Explainability
- Credit Risk Scoring System
- Business Interpretation & Conclusion



1. Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
%matplotlib inline

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    classification_report, roc_curve
)
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    BaggingClassifier, AdaBoostClassifier,
    RandomForestClassifier, GradientBoostingClassifier
)
from xgboost import XGBClassifier
import shap
import warnings
warnings.filterwarnings('ignore')

# — Global aesthetics —
sns.set_style('darkgrid')
PALETTE = ['#2ecc71', '#e74c3c', '#3498db', '#9b59b6', '#f39c12', '#1abc9c']
plt.rcParams.update({'figure.dpi': 110, 'font.size': 11})

kFold = StratifiedKFold(n_splits=5)
SEED = 101
print('Libraries loaded successfully.')
```

Libraries loaded successfully.

2. Loading & Understanding the Dataset

```
from google.colab import files
```

```
uploaded = files.upload()
```

loan_data.csv

loan_data.csv(text/csv) - 751253 bytes, last modified: 6/8/2026 - 100% done
Saving loan_data.csv to loan_data.csv

```
df = pd.read_csv('loan_data.csv')
print(f'Shape: {df.shape}')
df.head()
```

Shape: (9578, 14)

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util
0	1	debt_consolidation	0.1189	829.10	11.350407	19.48	737	5639.958333	28854	52.1
1	1	credit_card	0.1071	228.22	11.082143	14.29	707	2760.000000	33623	76.7
2	1	debt_consolidation	0.1357	366.86	10.373491	11.63	682	4710.000000	3511	25.6
3	1	debt_consolidation	0.1008	162.34	11.350407	8.10	712	2699.958333	33667	73.2
4	1	credit_card	0.1426	102.92	11.299732	14.97	667	4066.000000	4740	39.5

Next steps:

Generate code with df

New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   credit.policy          9578 non-null   int64
1   purpose                9578 non-null   object
2   int.rate               9578 non-null   float64
3   installment            9578 non-null   float64
4   log.annual.inc         9578 non-null   float64
5   dti                    9578 non-null   float64
6   fico                   9578 non-null   int64
7   days.with.cr.line      9578 non-null   float64
8   revol.bal              9578 non-null   int64
9   revol.util             9578 non-null   float64
10  inq.last.6mths         9578 non-null   int64
11  delinq.2yrs            9578 non-null   int64
12  pub.rec                9578 non-null   int64
13  not.fully.paid         9578 non-null   int64
dtypes: float64(6), int64(7), object(1)
memory usage: 1.0+ MB
```

```
df.describe()
```

	credit.policy	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util
count	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000	9578.000000	9.578000e+03	9578.000000
mean	0.804970	0.122640	319.089413	10.932117	12.606679	710.846314	4560.767197	1.691396e+04	46.799
std	0.396245	0.026847	207.071301	0.614813	6.883970	37.970537	2496.930377	3.375619e+04	29.014
min	0.000000	0.060000	15.670000	7.547502	0.000000	612.000000	178.958333	0.000000e+00	0.000
25%	1.000000	0.103900	163.770000	10.558414	7.212500	682.000000	2820.000000	3.187000e+03	22.600
50%	1.000000	0.122100	268.950000	10.928884	12.665000	707.000000	4139.958333	8.596000e+03	46.300
75%	1.000000	0.140700	432.762500	11.291293	17.950000	737.000000	5730.000000	1.824950e+04	70.900
max	1.000000	0.216400	940.140000	14.528354	29.960000	827.000000	17639.958330	1.207359e+06	119.000

```
# Null value check
null_total = df.isnull().sum().sum()
print(f'Total null values: {null_total}')
print('\nTarget class distribution:')
print(df['not.fully.paid'].value_counts())
print(f'\nDefault rate: {df["not.fully.paid"].mean():.2%}')
```

```
Total null values: 0

Target class distribution:
not.fully.paid
0      8045
1      1533
Name: count, dtype: int64
```

Default rate: 16.01%

Mr F

3. Encoding Categorical Features

The `purpose` column contains 6 categorical loan types. We apply **Label Encoding** to convert them into integer codes compatible with all downstream models.

```
print('Unique purposes:', df['purpose'].value_counts().to_dict())
df['purpose'] = LabelEncoder().fit_transform(df['purpose'])
print('\nEncoding complete. Sample:')
df.head()
```

Unique purposes: {'debt_consolidation': 3957, 'all_other': 2331, 'credit_card': 1262, 'home_improvement': 629, 'small_business':

Encoding complete. Sample:

	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util	inq.last
0	1	2	0.1189	829.10	11.350407	19.48	737	5639.958333	28854	52.1	
1	1	1	0.1071	228.22	11.082143	14.29	707	2760.000000	33623	76.7	
2	1	2	0.1357	366.86	10.373491	11.63	682	4710.000000	3511	25.6	
3	1	2	0.1008	162.34	11.350407	8.10	712	2699.958333	33667	73.2	
4	1	1	0.1426	102.92	11.299732	14.97	667	4066.000000	4740	39.5	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

4. Exploratory Data Analysis

Visualisations below explore the key relationships between borrower attributes and default risk.

```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

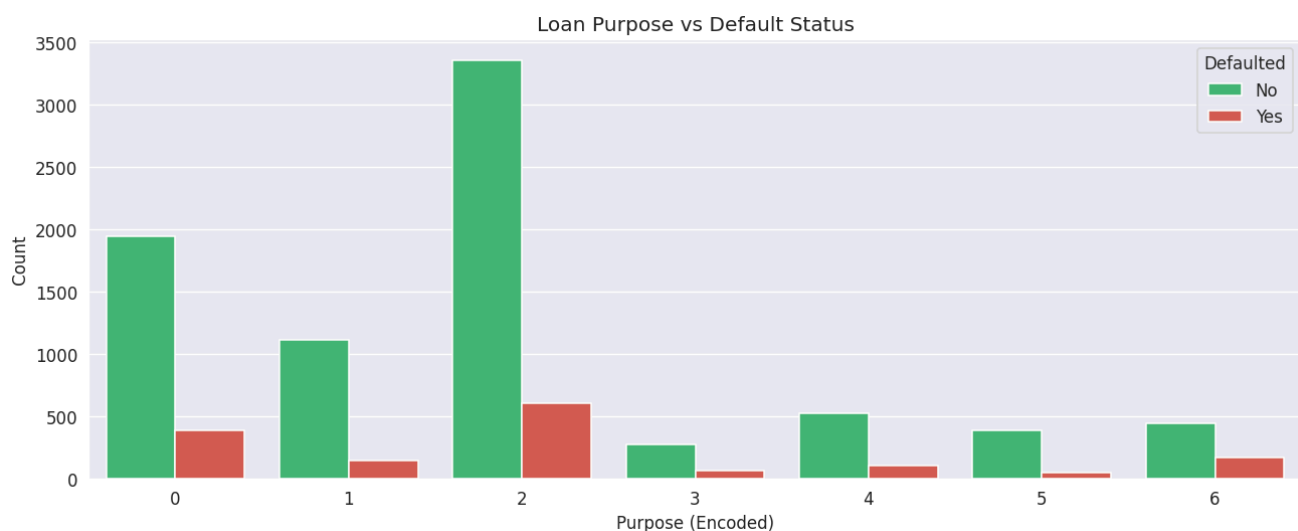
# FICO by credit policy
axes[0].hist(df['fico'][df['credit.policy']==1], bins=30, alpha=0.7, label='Credit Policy = 1', color='#2ecc71')
axes[0].hist(df['fico'][df['credit.policy']==0], bins=30, alpha=0.7, label='Credit Policy = 0', color='#e74c3c')
axes[0].set_xlabel('FICO Score'); axes[0].set_ylabel('Count')
axes[0].set_title('FICO Distribution by Credit Policy')
axes[0].legend()

# FICO by repayment
df[df['not.fully.paid']==1]['fico'].hist(bins=30, alpha=0.6, color='#e74c3c', label='Defaulted', ax=axes[1])
df[df['not.fully.paid']==0]['fico'].hist(bins=30, alpha=0.6, color='#2ecc71', label='Repaid', ax=axes[1])
axes[1].set_xlabel('FICO Score'); axes[1].set_ylabel('Count')
axes[1].set_title('FICO Distribution by Repayment Status')
axes[1].legend()

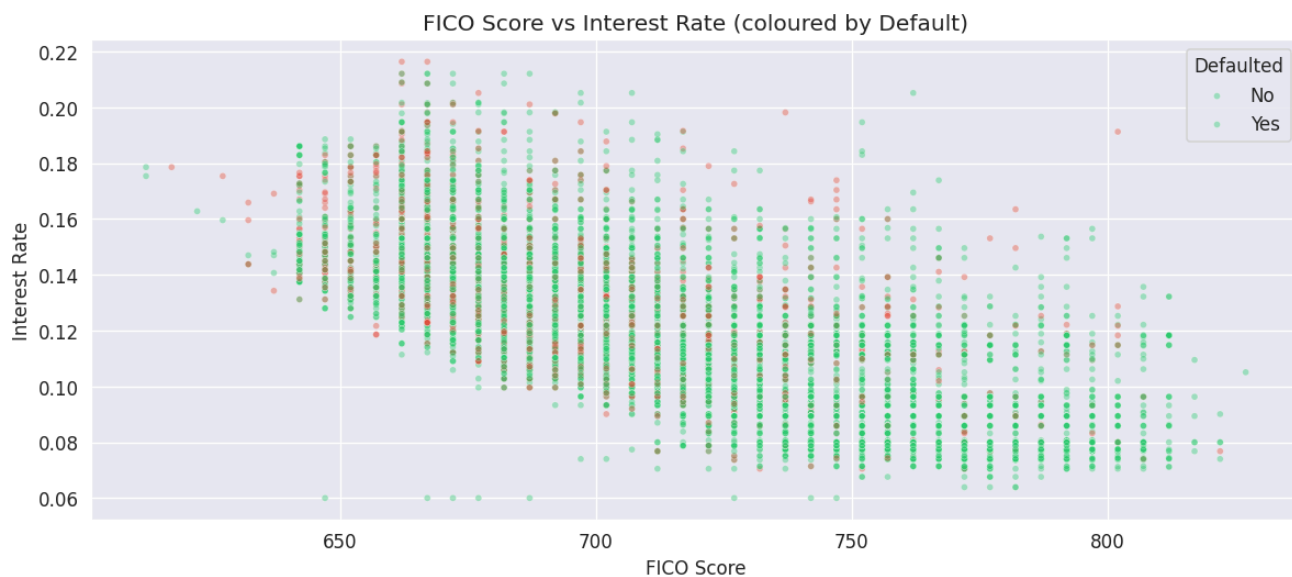
plt.tight_layout()
plt.show()
```



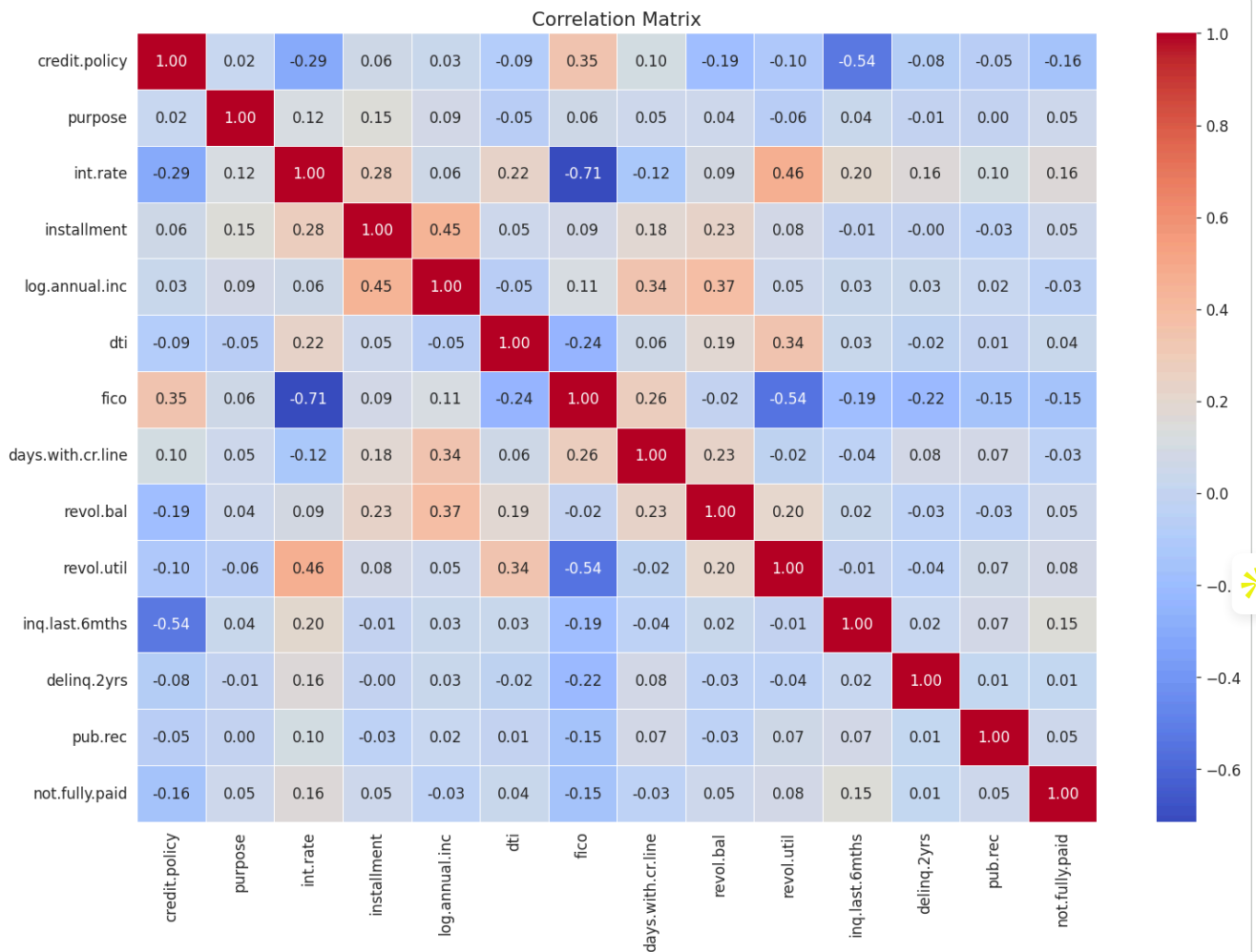
```
plt.figure(figsize=(12, 5))
sns.countplot(data=df, x='purpose', hue='not.fully.paid', palette=['#2ecc71', '#e74c3c'])
plt.title('Loan Purpose vs Default Status')
plt.xlabel('Purpose (Encoded)')
plt.ylabel('Count')
plt.legend(title='Defaulted', labels=['No', 'Yes'])
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(11, 5))
sns.scatterplot(x='fico', y='int.rate', data=df, hue='not.fully.paid',
                palette=['#2ecc71', '#e74c3c'], alpha=0.4, s=15)
plt.title('FICO Score vs Interest Rate (coloured by Default)')
plt.xlabel('FICO Score')
plt.ylabel('Interest Rate')
plt.legend(title='Defaulted', labels=['No', 'Yes'])
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(14, 10))
sns.heatmap(df.corr(), cmap='coolwarm', annot=True, fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix', fontsize=14)
plt.tight_layout()
plt.show()
```



5. Train / Test Split

A 70/30 stratified split is used to ensure consistent class proportions across both sets. **All models share this exact split** — no resampling is performed elsewhere.

```
X = df.drop('not.fully.paid', axis=1)
y = df['not.fully.paid']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=SEED, stratify=y
)
print(f'Train size : {X_train.shape[0]} samples')
print(f'Test size : {X_test.shape[0]} samples')
print(f'Default rate (train): {y_train.mean():.2%}')
print(f'Default rate (test) : {y_test.mean():.2%}')
```

```
Train size : 6704 samples
Test size : 2874 samples
Default rate (train): 16.01%
Default rate (test) : 16.01%
```

6. Evaluation Helper

A single `evaluate_model` function ensures every model is assessed on the same 5 metrics: Accuracy, Precision, Recall, F1, and ROC-AUC.

```
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te, results_dict):
    """
    Train model, compute metrics, print confusion matrix + classification report,
    and append results to results_dict.
    """
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[:,1]

    acc = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred, zero_division=0)
    rec = recall_score(y_te, y_pred, zero_division=0)
    f1 = f1_score(y_te, y_pred, zero_division=0)
    auc = roc_auc_score(y_te, y_proba)

    print(f'{' '*60}')
    print(f' {name}')
    print(f'{' '*60}')
    print('Confusion Matrix:')
    print(confusion_matrix(y_te, y_pred))
    print('\nClassification Report:')
    print(classification_report(y_te, y_pred))
    print(f'Accuracy : {acc:.4f}')
    print(f'Precision: {prec:.4f}')
    print(f'Recall : {rec:.4f}')
    print(f'F1 Score : {f1:.4f}')
    print(f'ROC-AUC : {auc:.4f}\n')

    results_dict[name] = {
        'Accuracy' : round(acc, 4),
        'Precision' : round(prec, 4),
        'Recall' : round(rec, 4),
        'F1 Score' : round(f1, 4),
        'ROC-AUC' : round(auc, 4),
    }
    return model

results = {} # Accumulates all model metrics
print('Evaluation helper ready.')
```

Evaluation helper ready.

7. Model Training & Evaluation

Each model is trained once on `X_train / y_train` and evaluated on the held-out `X_test / y_test`. Results are stored in `results` for the unified comparison table.

7.1 Decision Tree (Grid-Search Tuned)

```
# =====
# 7.1 Decision Tree (Grid-Search Tuned)
# =====

dt_base = DecisionTreeClassifier(
    random_state=SEED,
    class_weight='balanced'
)

param_grid = {
    'max_depth': [3, 5, 7, 10, 15, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5]
}

gs = GridSearchCV(
    estimator=dt_base,
```

```

        param_grid=param_grid,
        scoring='f1',
        cv=kFold,
        n_jobs=-1
    )

gs.fit(X_train, y_train)

best_params = gs.best_params_

print("Best Parameters:")
print(best_params)

dt_clf = DecisionTreeClassifier(
    **best_params,
    random_state=SEED,
    class_weight='balanced'
)

dt_clf = evaluate_model(
    'Decision Tree',
    dt_clf,
    X_train,
    y_train,
    X_test,
    y_test,
    results
)

```

Best Parameters:
{'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}

=====
Decision Tree
=====

Confusion Matrix:

```
[[1922  492]
 [ 264  196]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.80	0.84	2414
1	0.28	0.43	0.34	460
accuracy			0.74	2874
macro avg	0.58	0.61	0.59	2874
weighted avg	0.78	0.74	0.76	2874

Accuracy : 0.7370

Precision: 0.2849

Recall : 0.4261

F1 Score : 0.3415

ROC-AUC : 0.6459

7.2 Bagging Classifier

```

# =====
# 7.2 Bagging Classifier
# =====

bag_clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(
        **best_params,
        random_state=SEED,
        class_weight='balanced'
    ),
    n_estimators=100,
    bootstrap=True,
    random_state=SEED,
    n_jobs=-1
)

bag_clf = evaluate_model(
    'Bagging',
    bag_clf,
    X_train,
    y_train,

```

```

X_test,
y_test,
results
)

```

```

=====
Bagging
=====
Confusion Matrix:
[[1822  592]
 [ 244  216]]

Classification Report:
              precision    recall  f1-score   support

     0       0.88       0.75       0.81       2414
     1       0.27       0.47       0.34       460

 accuracy          0.71       2874
 macro avg         0.57       2874
weighted avg         0.78       2874

Accuracy : 0.7091
Precision: 0.2673
Recall   : 0.4696
F1 Score : 0.3407
ROC-AUC  : 0.6684

```

7.3 AdaBoost

```

ada_clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=2),
    learning_rate=0.5, n_estimators=200, random_state=SEED
)
ada_clf = evaluate_model('AdaBoost', ada_clf, X_train, y_train, X_test, y_test, results)

```

```

=====
AdaBoost
=====
Confusion Matrix:
[[2395  19]
 [ 454   6]]

Classification Report:
              precision    recall  f1-score   support

     0       0.84       0.99       0.91       2414
     1       0.24       0.01       0.02       460

 accuracy          0.84       2874
 macro avg         0.54       2874
weighted avg         0.74       2874

Accuracy : 0.8354
Precision: 0.2400
Recall   : 0.0130
F1 Score : 0.0247
ROC-AUC  : 0.6604

```

7.4 Random Forest

```

# =====
# 7.4 Random Forest
# =====

rf_clf = RandomForestClassifier(
    n_estimators=300,
    class_weight='balanced',
    random_state=SEED,
    n_jobs=-1
)

rf_clf = evaluate_model(
    'Random Forest',
    rf_clf,
    X_train,

```

```

y_train,
X_test,
y_test,
results
)

```

```

=====
Random Forest
=====
Confusion Matrix:
[[2410   4]
 [ 456   4]]

Classification Report:
      precision    recall  f1-score   support

     0       0.84      1.00      0.91      2414
     1       0.50      0.01      0.02       460

   accuracy       0.84      0.84      0.84      2874
  macro avg       0.67      0.50      0.46      2874
weighted avg       0.79      0.84      0.77      2874

Accuracy : 0.8399
Precision: 0.5000
Recall   : 0.0087
F1 Score : 0.0171
ROC-AUC  : 0.6481

```

7.5 Gradient Boosting

```

gb_clf = GradientBoostingClassifier(learning_rate=0.05, n_estimators=300, random_state=SEED)
gb_clf = evaluate_model('Gradient Boosting', gb_clf, X_train, y_train, X_test, y_test, results)

```

```

=====
Gradient Boosting
=====
Confusion Matrix:
[[2383  31]
 [ 448  12]]

Classification Report:
      precision    recall  f1-score   support

     0       0.84      0.99      0.91      2414
     1       0.28      0.03      0.05       460

   accuracy       0.83      0.83      0.83      2874
  macro avg       0.56      0.51      0.48      2874
weighted avg       0.75      0.83      0.77      2874

Accuracy : 0.8333
Precision: 0.2791
Recall   : 0.0261
F1 Score : 0.0477
ROC-AUC  : 0.6564

```

7.6 XGBoost

XGBoost applies regularised gradient boosting with built-in handling of class imbalance via `scale_pos_weight`. The parameter is set to the majority/minority class ratio, giving the model stronger signal for the rare default class.

```

scale = (y_train == 0).sum() / (y_train == 1).sum()
xgb_clf = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale,
    eval_metric='logloss',
    random_state=SEED,
    n_jobs=-1
)
xgb_clf = evaluate_model('XGBoost', xgb_clf, X_train, y_train, X_test, y_test, results)

```

```
=====
XGBoost
=====
Confusion Matrix:
[[1897  517]
 [ 298  162]]

Classification Report:
              precision    recall  f1-score   support

      0       0.86       0.79       0.82       2414
      1       0.24       0.35       0.28        460

   accuracy       0.72       0.72       0.72       2874
  macro avg       0.55       0.57       0.55       2874
weighted avg       0.76       0.72       0.74       2874

Accuracy : 0.7164
Precision: 0.2386
Recall   : 0.3522
F1 Score : 0.2845
ROC-AUC  : 0.6282
```

8. Unified Model Comparison Table



The table below consolidates all six models on five evaluation metrics. The best value in each column is highlighted in **green**.

```
comparison_df = pd.DataFrame(results).T
comparison_df.index.name = 'Model'

def highlight_best(col):
    is_best = col == col.max()
    return ['background-color: #d5f5e3; font-weight: bold' if v else '' for v in is_best]

comparison_df.style\
    .apply(highlight_best)\
    .format('{:.4f}')\
    .set_caption('Model Comparison - Credit Risk Assessment')\
    .set_table_styles([{'selector': 'caption',
                        'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
```

Model Comparison — Credit Risk Assessment					
	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Model					
Decision Tree	0.7370	0.2849	0.4261	0.3415	0.6459
Bagging	0.7091	0.2673	0.4696	0.3407	0.6684
AdaBoost	0.8354	0.2400	0.0130	0.0247	0.6604
Random Forest	0.8399	0.5000	0.0087	0.0171	0.6481
Gradient Boosting	0.8333	0.2791	0.0261	0.0477	0.6564
XGBoost	0.7164	0.2386	0.3522	0.2845	0.6282

```
best_model_name = comparison_df['ROC-AUC'].idxmax()
print(f'Best model by ROC-AUC: {best_model_name}')
print(comparison_df.loc[best_model_name])
```

```
Best model by ROC-AUC: Bagging
Accuracy    0.7091
Precision   0.2673
Recall      0.4696
F1 Score    0.3407
ROC-AUC     0.6684
Name: Bagging, dtype: float64
```

9. ROC Curve Comparison

The ROC curve measures a model's ability to distinguish between defaulters and repayers across all probability thresholds. Higher AUC → better discriminative power.

```

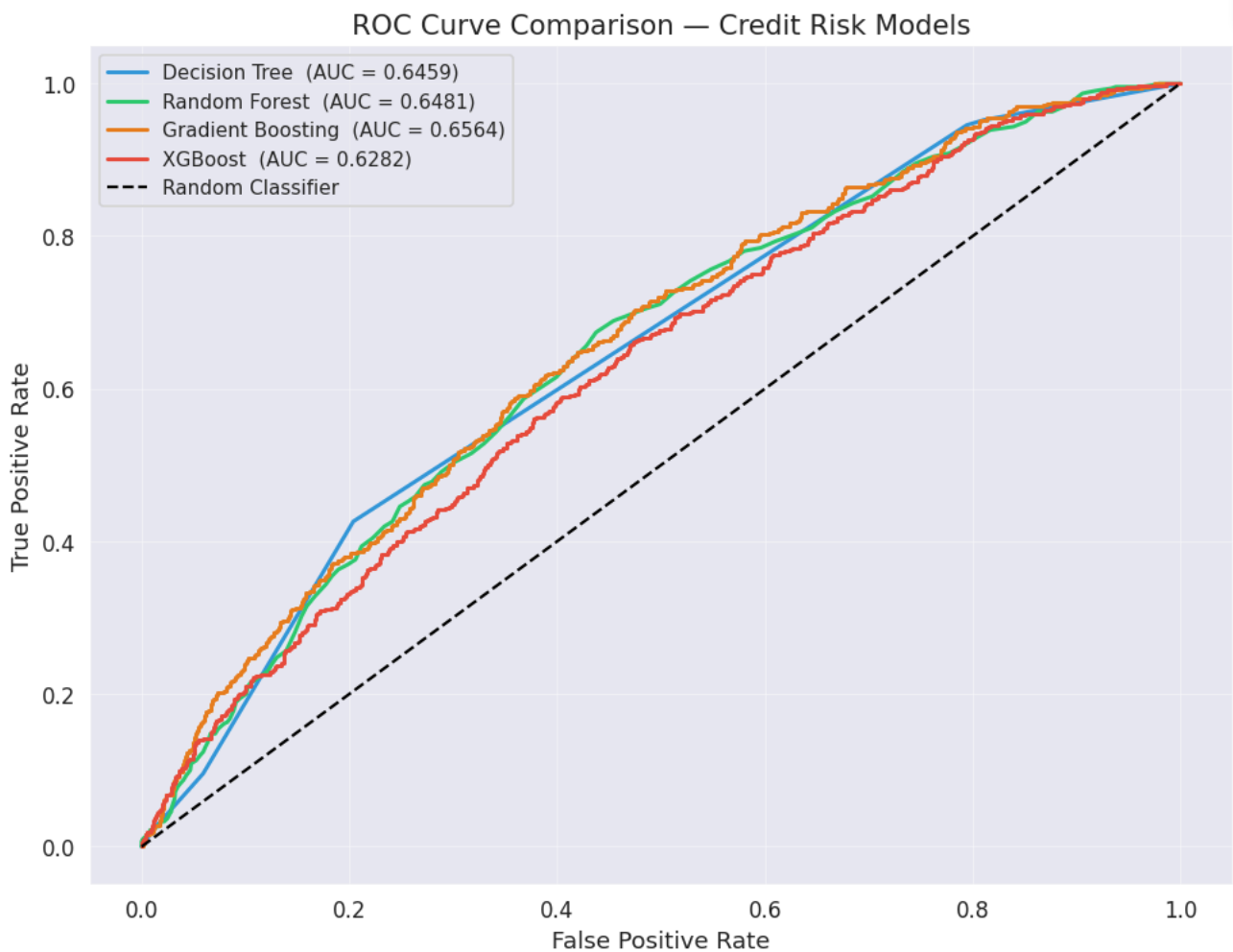
roc_models = {
    'Decision Tree' : dt_clf,
    'Random Forest' : rf_clf,
    'Gradient Boosting' : gb_clf,
    'XGBoost' : xgb_clf,
}

fig, ax = plt.subplots(figsize=(9, 7))
colors = ['#3498db', '#2ecc71', '#e67e22', '#e74c3c']

for (name, model), color in zip(roc_models.items(), colors):
    y_proba = model.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    auc = roc_auc_score(y_test, y_proba)
    ax.plot(fpr, tpr, label=f'{name} (AUC = {auc:.4f})', color=color, lw=2)

ax.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Classifier')
ax.set_xlabel('False Positive Rate', fontsize=12)
ax.set_ylabel('True Positive Rate', fontsize=12)
ax.set_title('ROC Curve Comparison — Credit Risk Models', fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()

```



10. Feature Importance Analysis

We extract feature importances from the best tree-based model (XGBoost). Importance values reflect how much each feature contributes to reducing impurity across all trees.

Key drivers to watch:

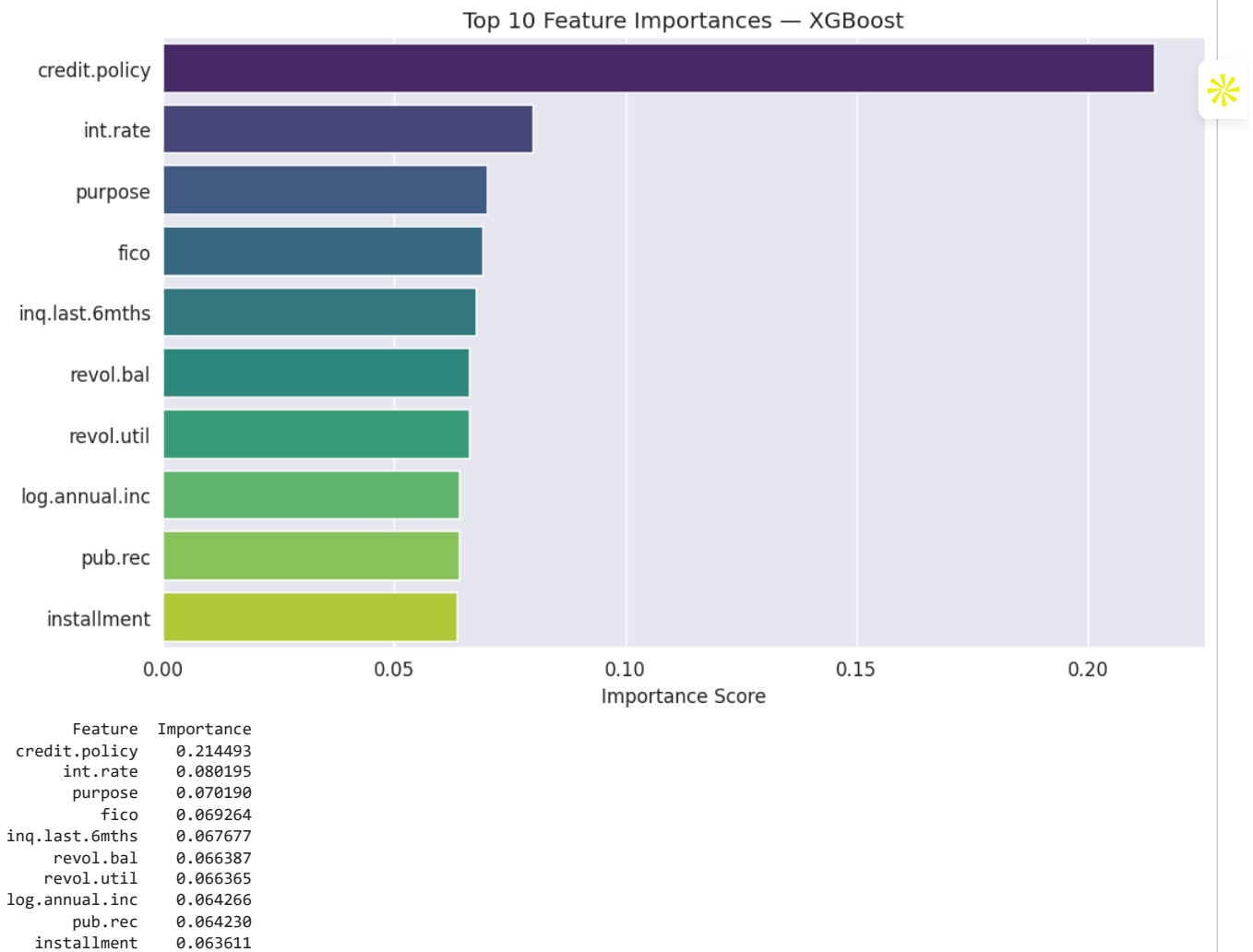
- **FICO Score** — Higher FICO scores signal creditworthiness; lower scores correlate with default.
- **Interest Rate** (`int.rate`) — Higher rates are typically assigned to riskier borrowers, making rate a proxy for default risk.

- **DTI (Debt-to-Income)** — A high DTI suggests the borrower is over-leveraged and more likely to struggle.
- **Revolving Utilisation (`revol.util`)** — Heavy use of revolving credit lines signals financial stress.
- **Log Annual Income** — Higher income buffers against default.

```
feature_names = X.columns.tolist()
importances = xgb_clf.feature_importances_
feat_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})\
    .sort_values('Importance', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(data=feat_df, x='Importance', y='Feature', palette='viridis')
plt.title('Top 10 Feature Importances — XGBoost', fontsize=13)
plt.xlabel('Importance Score')
plt.ylabel('')
plt.tight_layout()
plt.show()

print(feat_df.to_string(index=False))
```



11. SHAP Explainability

SHAP (SHapley Additive exPlanations) distributes the model's prediction for each observation across its input features — analogous to Shapley values in cooperative game theory.

Unlike feature importance scores, SHAP provides:

- **Direction** — does a high FICO *increase* or *decrease* default probability?
- **Magnitude** — by how much?

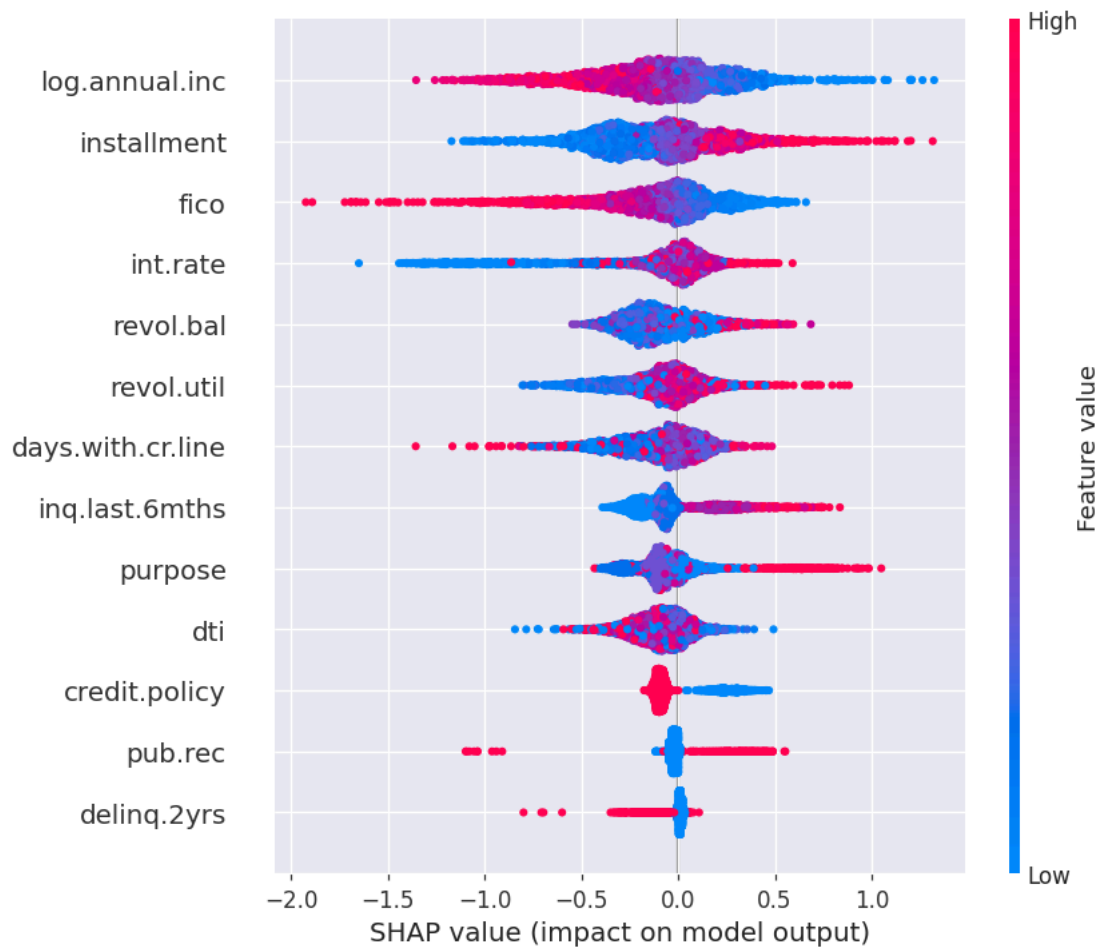
- **Per-observation** explanations — not just global averages.

The **Summary Plot** (beeswarm) shows SHAP values for every test observation, coloured by feature value. The **Bar Plot** ranks features by mean absolute SHAP value — a robust importance measure.

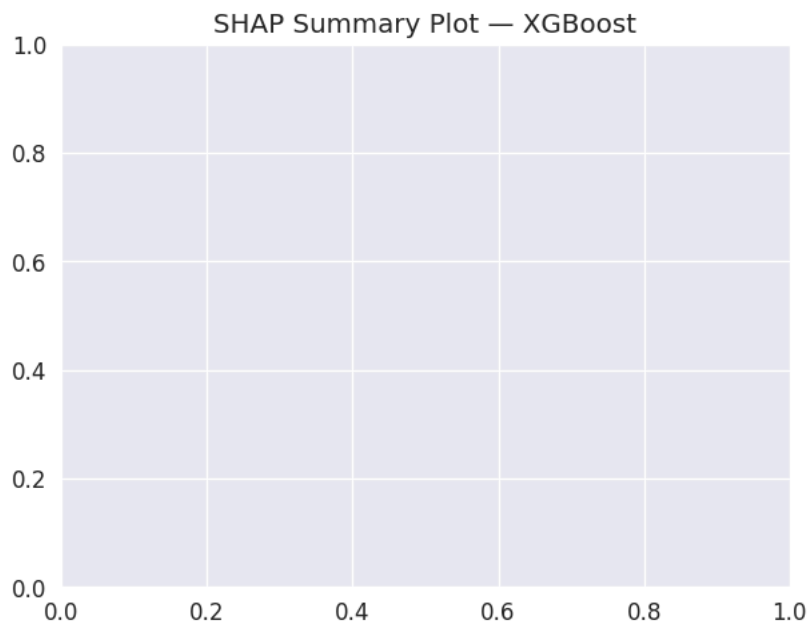
```
explainer = shap.TreeExplainer(xgb_clf)
shap_values = explainer.shap_values(X_test)

# Summary (beeswarm) plot
plt.figure()
shap.summary_plot(shap_values, X_test, plot_type='dot', show=True)
plt.title('SHAP Summary Plot - XGBoost', fontsize=13)
```

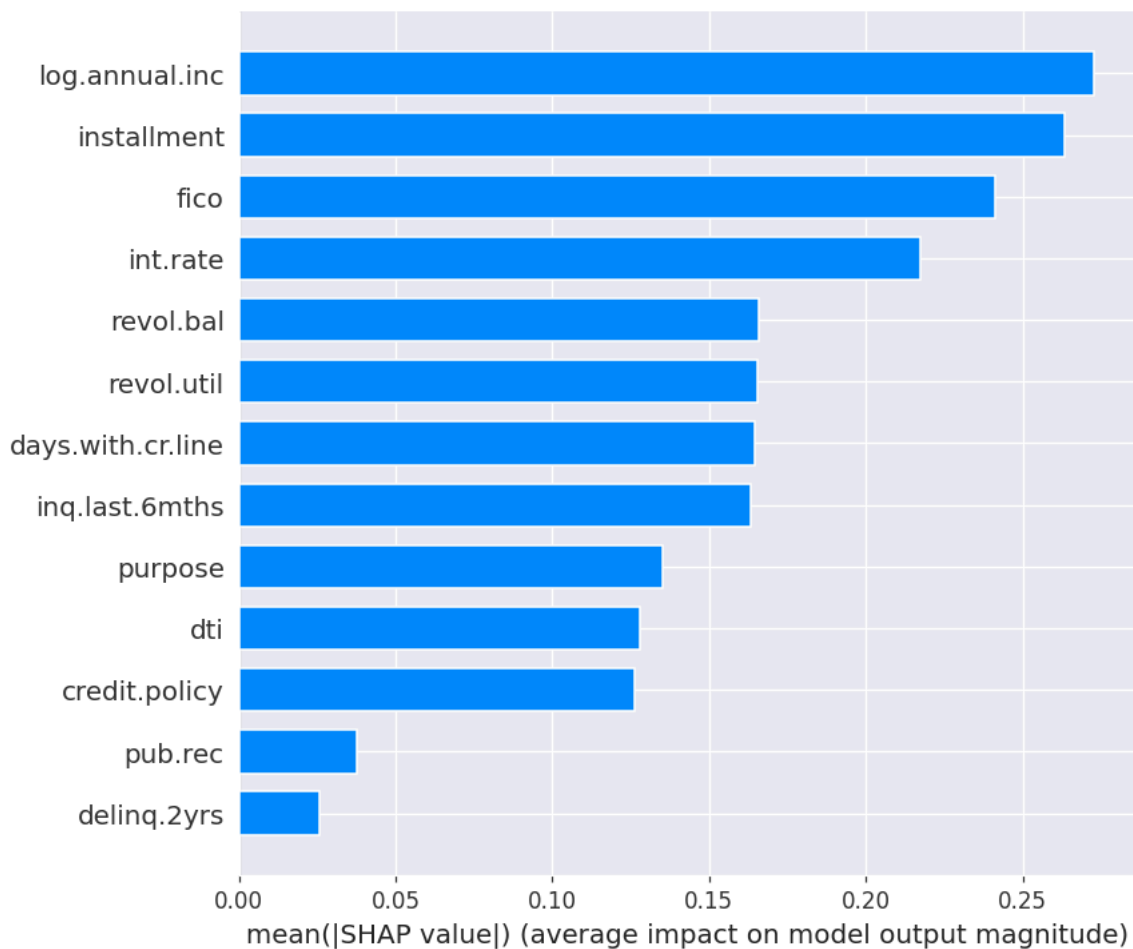




Text(0.5, 1.0, 'SHAP Summary Plot — XGBoost')



```
# Bar plot — mean |SHAP|
plt.figure()
shap.summary_plot(shap_values, X_test, plot_type='bar', show=True)
```



Interpretation

- **High FICO** → large negative SHAP values → *reduces* default probability. ✓
- **High interest rate** → large positive SHAP values → *increases* default risk. ⚠
- **High DTI / revolving utilisation** → positive SHAP → elevated risk. ⚠
- **High log annual income** → negative SHAP → borrower better positioned to repay. ✓
- **Recent credit enquiries** (`inq.last.6mths`) → positive SHAP → desperation signal. ⚠

These findings are consistent with established credit-risk literature and support using this model as a transparent, auditable tool for lending decisions.

12. Credit Risk Score System

We transform raw default probabilities into a standardised **Risk Score (0–100)**:

Risk Score = default_probability × 100

Risk Score	Risk Category
0 – 30	Low Risk
31 – 70	Medium Risk
71 – 100	High Risk

This scoring system mirrors the output of commercial bureau models (e.g. FICO, VantageScore) and enables portfolio segmentation, limit setting, and pricing decisions.

```
default_proba = xgb_clf.predict_proba(X_test)[: , 1]
risk_score    = (default_proba * 100).round(1)

def assign_risk_category(score):
    if score <= 30: return 'Low Risk'
    elif score <= 70: return 'Medium Risk'
```

```

else:
    return 'High Risk'

risk_df = pd.DataFrame({
    'Actual Outcome' : y_test.values,
    'Predicted Outcome' : xgb_clf.predict(X_test),
    'Default Probability': default_proba.round(4),
    'Risk Score' : risk_score,
    'Risk Category' : [assign_risk_category(s) for s in risk_score]
}).reset_index(drop=True)

print('Sample risk assessments:')
risk_df.head(20)

```

Sample risk assessments:

	Actual Outcome	Predicted Outcome	Default Probability	Risk Score	Risk Category
0	0	0	0.4656	46.599998	Medium Risk
1	1	0	0.4887	48.900002	Medium Risk
2	0	0	0.1756	17.600000	Low Risk
3	0	0	0.2391	23.900000	Low Risk
4	0	0	0.2334	23.299999	Low Risk
5	0	0	0.4590	45.900002	Medium Risk
6	0	0	0.3177	31.799999	Medium Risk
7	1	0	0.0433	4.300000	Low Risk
8	1	0	0.3573	35.700001	Medium Risk
9	0	0	0.3478	34.799999	Medium Risk
10	0	0	0.3785	37.799999	Medium Risk
11	0	0	0.2986	29.900000	Low Risk
12	0	0	0.3737	37.400002	Medium Risk
13	0	0	0.3782	37.799999	Medium Risk
14	0	0	0.4891	48.900002	Medium Risk
15	0	0	0.0862	8.600000	Low Risk
16	0	0	0.2090	20.900000	Low Risk
17	0	1	0.7012	70.099998	High Risk
18	0	0	0.0715	7.200000	Low Risk
19	0	0	0.3179	31.799999	Medium Risk

Next steps:

[Generate code with risk_df](#)

[New interactive sheet](#)

```

# Distribution of risk categories
cat_counts = risk_df['Risk Category'].value_counts()
colors_risk = ['#2ecc71', '#f39c12', '#e74c3c']

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

cat_counts.plot(kind='bar', ax=axes[0], color=colors_risk, edgecolor='white', rot=0)
axes[0].set_title('Risk Category Distribution (Test Set)', fontsize=13)
axes[0].set_ylabel('Count')

axes[1].hist(risk_score, bins=40, color='#3498db', edgecolor='white', alpha=0.85)
axes[1].axvline(30, color='#2ecc71', linestyle='--', lw=2, label='Low/Medium boundary')
axes[1].axvline(70, color='#e74c3c', linestyle='--', lw=2, label='Medium/High boundary')
axes[1].set_xlabel('Risk Score')
axes[1].set_ylabel('Count')
axes[1].set_title('Risk Score Histogram', fontsize=13)
axes[1].legend()

plt.tight_layout()
plt.show()

print('\nActual default rate per risk tier:')
print(risk_df.groupby('Risk Category')['Actual Outcome'].mean().rename('Default Rate').map('{:.2%}'.format))

```